

查找与有序索引

维护多少秩序，才能换取多少查询能力

408 · 数据结构 Topic DS-09

母问题：查询加速从哪里来，又由谁付费？

有序索引的生成链是

Order/Index → Search Reduction → Update Repair → Height Invariant → Cost Tradeoff.

顺序查找几乎不预处理；折半查找要求随机访问和有序区间；BST 把次序放进树路径；平衡树用旋转或分裂把高度约束在对数级。查询越强，通常越要在插入、删除、空间或维护时间上付费。

本册定位与 Owner 边界

- **Owns**：顺序/分块/折半查找、BST、AVL/红黑树的有序性与高度维护、B 树的查找与插删、B+ 树的查找结构。
- **Uses**：Atlas Foundation 成本向量；DS-B02 负责按 workload 比较有序索引与 Hash。
- **Stops**：Hash 映射归 DS10；磁盘块 I/O 的外部排序归 DS12；具体工程库实现只作 Extension。

目录

1	查找契约先固定	2
1.1	顺序、分块与折半：三种秩序预算	2
2	BST：次序下沉到路径	2
3	AVL 与红黑树：都控制高度，但修复契约不同	2
4	B 树：用节点扇出降低外存访问高度	2
5	B+ 树：内部节点导航，叶层拥有完整检索序列	3
6	代码与测试证据	3
7	复杂度与概念边界	6
8	做题控制协议	6

1 查找契约先固定

“查找”要先回答返回的是存在性、位置、前驱、首个重复 key 还是范围边界。折半查找依赖一个明确的区间不变量，例如目标若存在必在 $[low, high]$ 中；每轮比较后必须缩小区间，空区间才是失败证明。重复 key 若不固定取左界/右界，算法结果虽都可能“找到”却不满足接口。

1.1 顺序、分块与折半：三种秩序预算

顺序查找不要求数据有序，逐项排除候选。分块查找把表分成若干块，维护“块间有序、块内可无序”的两级结构：先在索引表定位可能的块，再在块内顺序查找。其平均成本可写成

$$ASL \approx ASL_{index} + ASL_{block},$$

因此块数增加会让索引阶段变长、块内阶段变短；不能只优化其中一项。折半查找则要求整体有序且支持随机访问，每轮用中点把候选区间近似减半。

三者逐步增加维护秩序以缩小查询候选：无索引、稀疏块索引、全局有序随机访问。若数据频繁插入，维持整体连续有序的移动成本可能抵消查询收益。

2 BST：次序下沉到路径

BST 对每个节点保持左子树 key 小于节点、右子树 key 大于节点（重复值策略需显式约定）。查找、插入、删除都沿一条根叶路径；复杂度是 $O(h)$ ，不是无条件 $O(\log n)$ 。退化链的高度为 n ，所以平衡条件不是装饰而是复杂度证明的前提。

删除的三种结构分支

叶节点直接删除；单孩子用孩子替代节点；双孩子从中序后继/前驱取替代 key，再在其原位置删除。每一步都要恢复局部 BST 有序性，不能只交换值而忘记释放或接回子树。

3 AVL 与红黑树：都控制高度，但修复契约不同

AVL 约束任一节点左右子树高度差不超过 1。插入后从修改点向根寻找第一个失衡祖先，根据较高子树方向形成 LL/RR/LR/RL；旋转后必须同时恢复 BST 中序有序和高度约束。

红黑树把严格高度平衡放宽为颜色约束：根与空叶为黑；红节点的孩子不能为红；从任一节点到其后代空叶的每条路径具有相同黑高。由此最长根叶路径不超过最短路径两倍，树高仍为 $O(\log n)$ 。插入冲突按叔节点颜色和结构方向决定变色或旋转；删除若移走黑色会产生黑高亏损，修复目标是把亏损上移、吸收或消除。

AVL 与红黑树不能共享旋转口诀

AVL 的触发量是高度差，红黑树的触发量是颜色与黑高。二者旋转都保持 BST 次序，但合法状态不同；只看到“失衡就旋转”无法判断变色与停止条件。

4 B 树：用节点扇出降低外存访问高度

设 m 阶 B 树，每个节点至多有 m 个孩子、至多 $m-1$ 个关键字；除根外的非叶节点至少有 $\lceil m/2 \rceil$ 个孩子，所有叶节点位于同一层。节点内关键字有序并划分孩子区间。查找先在节点内确定区间，再沿一个孩子下降；昂贵的是访问多少个节点/块。

插入总在叶层落位。节点溢出时把中间关键字上升父节点，左右关键字分成两个合法节点；父节点若再溢出就继续向上，根分裂会使树高增加 1。删除后若节点低于下限，先向兄弟借一个关键字并经父节点重分配；不能借才与兄弟及父分隔关键字合并。若根最终无关键字且只有一个孩子，用该孩子替代根，树高减 1。

统一不变量是：节点占用合法、节点内有序、孩子区间正确、所有叶同层。分裂和合并是在保持这些不变量下控制高度。

5 B+ 树：内部节点导航，叶层拥有完整检索序列

B+ 树把记录或记录指针集中在叶节点；非叶节点只保存导航分隔 key，关键字可能在内部和叶层重复。所有叶节点按 key 顺序链接，因此等值查找沿根到叶完成，范围查找定位起点后沿叶链扫描。

概念 A	≠ 概念 B	真正判据与题面信号	混淆后果
B 树	≠ B+ 树	B 树各层可命中记录；B+ 树内部只导航、叶链承载全体 key	误判成功层次和范围扫描路径
树的阶	≠ 节点关键字数	阶限制最大孩子数，关键字上限通常少 1	占用上下限算错
分裂	≠ 旋转借位	分裂制造新节点；借位在相邻节点和父分隔 key 间重分配	高度变化判断错

6 代码与测试证据

完整实现位于 code/ds09_ordered_index.hpp，测试位于 code/ds09_ordered_index_test.cpp。本册用边界明确的 BST 作为可执行核心，平衡树和 B/B+ 的分裂规则保留为机制扩展。

Listing 1: BST 查找、插入、删除与有序输出

```
12
13 class BinarySearchTree {
14 public:
15     bool contains(int key) const { return contains(root_.get(), key); }
16     void insert(int key) { insert(root_, key); }
17     bool erase(int key) { return erase(root_, key); }
18     std::vector<int> inorder() const { std::vector<int> result; inorder(
19         root_.get(), result); return result; }
20 private:
21     std::unique_ptr<Node> root_;
22     static bool contains(const Node* node, int key) { if (!node) return
23         false; if (key == node->key) return true; return key < node->key ?
24         contains(node->left.get(), key) : contains(node->right.get(), key); }
25     static void insert(std::unique_ptr<Node>& node, int key) { if (!node) {
26         node = std::make_unique<Node>(Node{key, nullptr, nullptr}); return; }
27         if (key < node->key) insert(node->left, key); else if (key > node->key
28             ) insert(node->right, key); }
```

```

24 static bool erase(std::unique_ptr<Node>& node, int key) {
25     if (!node) return false;
26     if (key < node->key) return erase(node->left, key);
27     if (key > node->key) return erase(node->right, key);
28     if (!node->left) { node = std::move(node->right); return true; }
29     if (!node->right) { node = std::move(node->left); return true; }
30     Node* successor = node->right.get(); while (successor->left)
    successor = successor->left.get();
31     node->key = successor->key; return erase(node->right, successor->key)
    ;
32 }
33 static void inorder(const Node* node, std::vector<int>& result) { if (!
    node) return; inorder(node->left.get(), result); result.push_back(node
    ->key); inorder(node->right.get(), result); }
34 };
35
36 class AvlTree {
37 public:
38     bool contains(int key) const { return contains(root_.get(), key); }
39     void insert(int key) { root_ = insert(std::move(root_), key); }
40     std::size_t height() const { return height(root_.get()); }
41     std::vector<int> inorder() const { std::vector<int> result; inorder(
        root_.get(), result); return result; }
42     bool invariant() const { int actual = 0; return check(root_.get(),
        nullptr, nullptr, actual); }
43
44 private:
45     struct AvlNode { int key; int height = 1; std::unique_ptr<AvlNode> left
        , right; };
46     std::unique_ptr<AvlNode> root_;
47     static int height(const AvlNode* node) { return node == nullptr ? 0 :
        node->height; }
48     static void refresh(AvlNode* node) { node->height = 1 + std::max(height
        (node->left.get()), height(node->right.get())); }
49     static int balance(const AvlNode* node) { return node == nullptr ? 0 :
        height(node->left.get()) - height(node->right.get()); }
50     static std::unique_ptr<AvlNode> rotate_right(std::unique_ptr<AvlNode>
        node) { auto pivot = std::move(node->left); node->left = std::move(
        pivot->right); refresh(node.get()); pivot->right = std::move(node);
        refresh(pivot.get()); return pivot; }
51     static std::unique_ptr<AvlNode> rotate_left(std::unique_ptr<AvlNode>
        node) { auto pivot = std::move(node->right); node->right = std::move(
        pivot->left); refresh(node.get()); pivot->left = std::move(node);
        refresh(pivot.get()); return pivot; }
52     static std::unique_ptr<AvlNode> insert(std::unique_ptr<AvlNode> node,
        int key) {
53         if (!node) return std::make_unique<AvlNode>(AvlNode{key, 1, nullptr,

```

```

    nullptr});
54     if (key < node->key) node->left = insert(std::move(node->left), key);
55     else if (key > node->key) node->right = insert(std::move(node->right)
, key);
56     else return node;
57     refresh(node.get()); const int factor = balance(node.get());
58     if (factor > 1 && key < node->left->key) return rotate_right(std:::
move(node));
59     if (factor < -1 && key > node->right->key) return rotate_left(std:::
move(node));
60     if (factor > 1 && key > node->left->key) { node->left = rotate_left(
std::move(node->left)); return rotate_right(std::move(node)); }
61     if (factor < -1 && key < node->right->key) { node->right =
rotate_right(std::move(node->right)); return rotate_left(std::move(
node)); }
62     return node;
63 }
64 static bool contains(const AvlNode* node, int key) { if (!node) return
false; if (key == node->key) return true; return key < node->key ?
contains(node->left.get(), key) : contains(node->right.get(), key); }
65 static void inorder(const AvlNode* node, std::vector<int>& result) { if
(!node) return; inorder(node->left.get(), result); result.push_back(
node->key); inorder(node->right.get(), result); }
66 static bool check(const AvlNode* node, const int* lower, const int*
upper, int& actual_height) {
67     if (!node) { actual_height = 0; return true; }
68     if ((lower && node->key <= *lower) || (upper && node->key >= *upper))
return false;
69     int left_height = 0, right_height = 0;
70     if (!check(node->left.get(), lower, &node->key, left_height) || !
check(node->right.get(), &node->key, upper, right_height)) return
false;
71     actual_height = 1 + std::max(left_height, right_height);
72     return node->height == actual_height && std::abs(left_height -
right_height) <= 1;
73 }
74 };
75 } // namespace ipara::ds09
76
77 #endif

```

Listing 2: 空树、重复 key 与删除分支测试

```

8  assert(!tree.contains(4));
9  tree.insert(5); tree.insert(3); tree.insert(7); tree.insert(6); tree.
    insert(8); tree.insert(3);
10  assert(tree.contains(6));
11  assert((tree.inorder() == std::vector<int>{3, 5, 6, 7, 8}));

```

```
12  assert(tree.erase(3)); assert(tree.erase(5)); assert(!tree.erase(42));
13  assert((tree.inorder() == std::vector<int>{6, 7, 8}));
14  ipara::ds09::AvlTree avl;
15  for (int key : {30, 20, 10, 25, 28, 40, 50}) { avl.insert(key); assert(
16      avl.invariant()); }
17  assert(avl.contains(28) && !avl.contains(99));
18  assert((avl.inorder() == std::vector<int>{10, 20, 25, 28, 30, 40, 50}))
19  ;
    assert(avl.height() <= 3);
}
```

```
clang++ -std=c++17 -Wall -Wextra -Werror -pedantic code/
    ds09_ordered_index_test.cpp -o /tmp/ds09_test
/tmp/ds09_test
```

7 复杂度与概念边界

结构/操作	平均或保证	最坏	依赖的不变量	维护代价
顺序查找	$O(n)$	$O(n)$	无序也可	几乎无预处理
折半查找	$O(\log n)$	$O(\log n)$	有序 + 随机访问	插入移动
BST	$O(h)$	$O(n)$	局部有序	可能退化
平衡树	$O(\log n)$	$O(\log n)$	高度/颜色约束	旋转/分裂

8 做题控制协议

先写返回契约和区间/树不变量，再看输入是否有序、是否支持随机访问、更新是否频繁。看到 Hash 适合精确等值但不适合范围查询；看到动态有序更新，才考虑平衡索引。每个复杂度都把高度或有序前提写出来。

压缩复原

五问：查询要返回什么？候选区间怎样缩小？秩序存在哪里？更新破坏哪个不变量？维护成本由谁承担？